- 理论作业
  - 1. 题目描述
  - 2. 代码展示

理论作业

1. 题目描述

1. 设计一个算法在带头结点的非空单链表L中第一个最大值结点（最大值结点可能有多个)之前插入一个值为x的结点。
2. 用带头结点的单链表表示整数集合，完成以下算法并分析时间复杂度： (1) 设计一个算法求两个集合A和B的差集运算，即C=A-B，要求算法的空间复杂度为O(1),并释放单链表A和B中不需要的结点。 (2) 假设集合中的元素按递增排列，设计一个高效算法求两个集合A和B的差集运算，即C=A-B，要求算法的空间复杂度为O(1),并释放单链表A和B中不需要的结点。

2. 代码展示

1.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct LNode {
    int data;
    struct LNode *next;
} LNode, *LinkList;

void InitList(LinkList *L, int arr[], int n) {
    *L = (LinkList)malloc(sizeof(LNode));
    (*L)->next = NULL;
    LNode *tail = *L;
    for (int i = 0; i < n; i++) {
        LNode *newNode = (LNode *)malloc(sizeof(LNode));
        newNode->data = arr[i];
        newNode->next = NULL;
        tail->next = newNode;
        tail = newNode;
    }
}
```

```c
void InsertBeforeMax(LinkList L, int x) {
    if (L == NULL || L->next == NULL) {
        printf("链表为空! \n");
        return;
    }

    LNode *p = L->next;
    int max = p->data;
    while (p != NULL) {
        if (p->data > max) {
            max = p->data;
        }
        p = p->next;
    }

    LNode *pre = L;
    p = L->next;
    while (p != NULL) {
        if (p->data == max) {
            break;
        }
        pre = p;
        p = p->next;
    }

    LNode *newNode = (LNode *)malloc(sizeof(LNode));
    newNode->data = x;
    newNode->next = p;
    pre->next = newNode;
}

void PrintList(LinkList L) {
    LNode *p = L->next;
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
    printf("\n");
}

int main() {
    int arr[] = {3, 7, 2, 7, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    LinkList L;
    InitList(&L, arr, n);

    printf("原链表: ");
    PrintList(L);

    int x = 9;
    InsertBeforeMax(L, x);

    printf("插入后: ");
    PrintList(L);

    return 0;
}
```

2.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct LNode {
    int data;
    struct LNode *next;
} LNode, *LinkList;

void InitList(LinkList *L, int arr[], int n) {
    *L = (LinkList)malloc(sizeof(LNode));
    (*L)->next = NULL;
    LNode *tail = *L;
    for (int i = 0; i < n; i++) {
        LNode *newNode = (LNode *)malloc(sizeof(LNode));
        newNode->data = arr[i];
        newNode->next = NULL;
        tail->next = newNode;
        tail = newNode;
    }
}

void PrintList(LinkList L) {
    LNode *p = L->next;
    if (p == NULL) {
        printf("空集\n");
        return;
    }
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
    printf("\n");
}

void FreeList(LinkList *L) {
    LNode *p = *L, *q;
    while (p != NULL) {
        q = p->next;
        free(p);
        p = q;
    }
    *L = NULL;
}

int IsExist(LinkList L, int x) {
    LNode *p = L->next;
    while (p != NULL) {
        if (p->data == x) return 1;
        p = p->next;
    }
    return 0;
}
```

```c
void DiffSet_Unordered(LinkList A, LinkList B) {
    LNode *pre = A;
    LNode *pa = A->next;
    LNode *temp;

    while (pa != NULL) {
        if (IsExist(B, pa->data)) {
            pre->next = pa->next;
            temp = pa;
            pa = pa->next;
            free(temp);
        } else {
            pre = pa;
            pa = pa->next;
        }
    }
    FreeList(&B);
}

void DiffSet_Ordered(LinkList A, LinkList B) {
    LNode *pre = A;
    LNode *pa = A->next;
    LNode *pb = B->next;
    LNode *temp;

    while (pa != NULL && pb != NULL) {
        if (pa->data < pb->data) {
            pre = pa;
            pa = pa->next;
        } else if (pa->data == pb->data) {
            pre->next = pa->next;
            temp = pa;
            pa = pa->next;
            free(temp);
        } else {
            pb = pb->next;
        }
    }
    FreeList(&B);
}

int main() {
    printf("===== 无序列表差集测试 =====\n");
    int arrA1[] = {3, 1, 5, 2, 7};
    int arrB1[] = {2, 5, 8};
    LinkList A1, B1;
    InitList(&A1, arrA1, 5);
    InitList(&B1, arrB1, 3);
    printf("原集合A: "); PrintList(A1);
    printf("原集合B: "); PrintList(B1);
    DiffSet_Unordered(A1, B1);
    printf("差集A-B: "); PrintList(A1);
    FreeList(&A1);

    printf("\n===== 有序列表差集测试 =====\n");
    int arrA2[] = {1, 3, 5, 7, 9};
```

```
    int arrB2[] = {3, 7, 8};
    LinkList A2, B2;
    InitList(&A2, arrA2, 5);
    InitList(&B2, arrB2, 3);
    printf("原集合A: "); PrintList(A2);
    printf("原集合B: "); PrintList(B2);
    DiffSet_Ordered(A2, B2);
    printf("差集A-B: "); PrintList(A2);
    FreeList(&A2);

    return 0;
}
```

时间复杂度分析

(1) 无序列表

- 遍历 A 的每个结点：设 A 长度为 n，B 长度为 m,，时间复杂度为 O(n×m)；

(2) 有序列表

- 双指针遍历 A 和 B，每个指针最多遍历一次：A 长度 n，B 长度 m，时间复杂度为 O(n+m)；